

Project 1: Data Heist at StreamBeats

A Python Data-Recovery and Analysis Challenge

Dr. Yaping Jing

The Scenario

StreamBeats, a small but fast-growing music-streaming startup, suffered a server crash last night. Their listening-history logs – the record of which listener played which song, when, and for how long – were only partially recovered. Some lines survived intact. Others are blank, truncated, or scrambled. The company has no idea which songs and artists are actually popular with their listeners anymore, and a major investor meeting is in two days.

Your team has been hired as **StreamBeats' junior data engineering squad**. Your job: rebuild a clean data model in Python, recover as much of the corrupted log as possible *without crashing on the bad lines*, analyze what you recover with NumPy and pandas, and deliver a short **Insights Report** that tells StreamBeats' leadership what their listeners actually like.

Team Formation

- Work in groups of **3 students**. Choose your own team name (something StreamBeats-themed is encouraged).
- Within your team, informally rotate through three roles as you move through the parts below, so everyone gets hands-on time with every skill area:
 - **Data Architect** – leads Part 1 (classes / data model)
 - **Recovery Engineer** – leads Part 2 (file parsing / error handling)
 - **Insights Analyst** – leads Part 3 (NumPy / pandas analysis)
- Rotating leads does *not* mean working in silos – all three of you should understand and be able to explain *every* part of the notebook. See *Academic Integrity* below.

Learning Objectives

By the end of this project, your team should be able to:

- Design a small set of cooperating classes to model real-world entities.
- Read messy, real-world-style data from a file, using `try/except` to recover gracefully from bad records instead of crashing.
- Convert recovered data into a pandas `DataFrame` and analyze it with NumPy and pandas.
- Communicate data-driven findings clearly to a non-technical audience.
- Coordinate a small collaborative software project and document each member's contribution.

1 Part 1: Rebuilding the Data Model

Before you can recover anything, you need somewhere to put it. Design and implement the following classes:

Task Define a class `Song` with attributes `title`, `artist`, `genre`, and `duration_sec`. Give it a method `summary()` that returns a one-line description (e.g. "Neon Tide by Kalea Voss (Synthpop, 3:42)").

Task Define a class `Listener` with attributes `listener_id`, `name`, and `home_city`. Make `listener_id` a **private** attribute, with a `get_listener_id()` method to access it.

Task Define a class `ListeningEvent` that links a `Listener` and a `Song` together, with attributes `listener`, `song`, `timestamp` (a string, e.g. "2026-03-14 21:05"), and `seconds_played`.

Task In a Markdown cell, briefly justify your design: why three separate classes instead of one big class, or instead of just using dictionaries?

2 Part 2: The Corrupted Archive

StreamBeats' surviving server log is a plain-text file, one listening event per line, in the format:

```
listener_id | listener_name | song_title | artist | timestamp | seconds_played
```

Unfortunately, the crash mangled the file: some lines are blank, some are missing fields, and some have a non-numeric value where `seconds_played` should be. A clean line looks like this:

```
1 L104 | Priya N. | Neon Tide | Kalea Voss | 2026-03-14 21:05 | 203
```

Task Run the data-generation script below *once* to create your team's own uniquely corrupted `streambeats_log.txt` file. Change `TEAM_SEED` to your team's name so your data – and your findings – differ from every other team's.

```
1 import random
2
3 TEAM_SEED = "change-me-to-your-team-name"
4 random.seed(TEAM_SEED)
5
6 songs = [
7     ("Neon Tide", "Kalea Voss", "Synthpop", 222),
8     ("Gravel Road", "Tomas Reyes", "Country", 198),
9     ("Static Bloom", "Nightjar", "Indie Rock", 251),
10    ("Midnight Ferry", "Yuna Cho", "R&B", 214),
11    ("Paper Lanterns", "The Lowlights", "Folk", 187),
12    ("Circuit Heart", "Kalea Voss", "Synthpop", 205),
13    ("Dust and Static", "Nightjar", "Indie Rock", 233),
14 ]
15 listeners = [("L101", "Priya N.", "Salisbury"), ("L102", "Marcus T.", "Ocean City"),
16             ("L103", "Wei L.", "Cambridge"), ("L104", "Ava R.", "Salisbury"),
17             ("L105", "Diego F.", "Easton")]
18
19 lines = []
20 for _ in range(400):
21     lid, lname, city = random.choice(listeners)
22     title, artist, genre, dur = random.choice(songs)
23     hour = random.randint(0, 23)
24     day = random.randint(1, 28)
25     played = random.randint(20, dur)
26     line = (f"{lid} | {lname} | {title} | {artist} | "
27            f"2026-03-{day:02d} {hour:02d}:00 | {played}")
28     lines.append(line)
29
30 # Corrupt ~15% of the lines to simulate crash damage
31 for _ in range(60):
32     i = random.randrange(len(lines))
33     kind = random.choice(["blank", "missing_field", "bad_number"])
34     if kind == "blank":
35         lines[i] = ""
```

```

36 elif kind == "missing_field":
37     parts = lines[i].split(" | ")
38     del parts[random.randrange(len(parts))]
39     lines[i] = " | ".join(parts)
40 else:
41     parts = lines[i].split(" | ")
42     parts[-1] = "N/A"
43     lines[i] = " | ".join(parts)
44
45 random.shuffle(lines)
46 with open("streambeats_log.txt", "w") as f:
47     for line in lines:
48         print(line, file=f)

```

Task Write a function `parse_line(line)` that splits one line on " | ", builds and returns a `ListeningEvent` object (using your Part 1 classes). It should **raise an exception** (e.g. let an `IndexError` or `ValueError` propagate) if the line is blank, has the wrong number of fields, or has a non-numeric `seconds_played`.

Task Using `with`, `open streambeats_log.txt` and loop over every line. For each line, call `parse_line()` inside a `try/except` block: on success, append the resulting `ListeningEvent` to a list called `events`; on failure, increment a counter of skipped lines instead of crashing.

Task Write a short **recovery report** to a new file, `recovery_report.txt`, using `with` and `print(..., file=...)`, stating: the total number of lines in the log, how many were successfully recovered, and how many were skipped.

3 Part 3: The Investigation

Now put NumPy and pandas to work on the `events` list you recovered.

Task Convert your list of `ListeningEvent` objects into a pandas `DataFrame` called `events_df`, with one column per attribute (`listener_name`, `song_title`, `artist`, `genre`, `timestamp`, `seconds_played`, etc.). (Hint: build a list of dictionaries first, then pass it to `pd.DataFrame()`.)

Task Using `.groupby()`, find **total listening time per genre** and display it sorted from highest to lowest.

Task Using boolean filtering (`==`, and `&/|` as needed), find all listening events for your single most-played artist.

Task Using NumPy, compute a **"popularity score"** for each song: convert each song's list of `seconds_played` values into a NumPy array and calculate its mean, then use vectorized arithmetic to express every song's average as a percentage of the single highest-scoring song's average. (This is the same kind of array-wide arithmetic from Lab 4's Vectorized Math section.)

Task Produce **two** charts directly from your `DataFrame` (e.g. `.plot.bar()`), such as: total listens by genre, and listening activity by hour of day (extracted from `timestamp`). Give each chart a title and axis labels.

Task Using `.pivot()`, reshape a subset of `events_df` so that day appears as the row index and genre appears as columns, with total `seconds_played` as the values.

4 Part 4: The Case File

In Markdown cells at the end of your notebook, write a short **Insights Report** for StreamBeats' leadership (non-technical readers – no code jargon):

Task State **three** concrete findings from Part 3, each backed by a specific number or chart (e.g. *"Synthpop accounted for 31% of all listening minutes, more than any other genre"*).

Task Report your **data-recovery rate** from Part 2 (e.g. “*We recovered 341 of 400 log entries – an 85% recovery rate*”), and briefly note what kinds of corruption you encountered.

Task Give StreamBeats **one** concrete recommendation based on your findings (e.g. which genre or artist to feature, or what time of day to schedule a release).

Grading Rubric

Component	Weight
Part 1 – Data model (classes)	15%
Part 2 – File recovery & error handling	25%
Part 3 – NumPy / pandas analysis	30%
Part 4 – Insights report	15%
Code quality & documentation	10%
Contribution log (see below)	5%

Academic Integrity and Teamwork

- Every team member must be able to explain *any* cell in the submitted notebook during in-class presentation.
- Include a short **contribution log** (a Markdown cell) listing which member led which part. Uneven contribution should be reported honestly; it will not by itself lower the group’s grade, but it informs individual grading if needed.
- Each team’s corrupted log file is unique (seeded by team name), so findings and exact numbers *should* differ between teams – identical results across teams will be investigated.

Submission Instructions

1. Make sure your notebook runs top-to-bottom without errors (*Kernel* → *Restart & Run All*).
2. One team member submits on behalf of the group.
3. Each team gives a 10-15 minutes presentation including a brief **demo** and of their findings.
4. Render your finished notebook as a PDF (e.g. *File* → *Download as* → *PDF via LaTeX*, or run `nbconvert` from the terminal: `jupyter nbconvert -to pdf teamname_project1.ipynb`).
5. Submit **both** of the following files, plus your generated `streambeats_log.txt` and `recovery_report.txt` files, to **MyClass/project1**:
 - `teamname_project1.ipynb`
 - `teamname_project1.pdf`(Replace `teamname` with your team’s chosen name, all lowercase, no spaces.)